

Memo de syntaxe

TP Controle des Risques Marche — SQL, pandas, numpy, PySpark

Perimetre de ce document. Reference *syntaxique*. Il contient les briques atomiques de chaque outil et leur correspondance d'un langage a l'autre. Il ne contient volontairement *aucun* des patterns composes qui constituent les missions : schema de reconciliation, selection de version courante, resolution du changement d'heure, detection d'unite, ventilation multi-periodes, decomposition de P&L. Ces assemblages sont l'exercice. Une brique n'est pas une solution. Une derniere page est vide pour que tu y consignes les patterns que tu construis. C'est cette page-la qui aura de la valeur en septembre, pas les precedentes.

0. Ouverture des sources

```
import sqlite3, duckdb
import numpy as np, pandas as pd

# SQLite
con = sqlite3.connect("data/risk.db")
df = pd.read_sql("SELECT * FROM trd_deal", con)
pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", con)

# CSV : les defaults sont rarement les bons
df = pd.read_csv(p, sep=";", decimal=".", encoding="latin-1",
                dtype={"deal_ref": str}, # jamais d'inference sur une cle
                parse_dates=["trade_dt"], dayfirst=True,
                keep_default_na=False, na_values=[""])

# Parquet (dossier partitionne : la cle de partition revient en colonne)
v = pd.read_parquet("data/volumes_hourly")
v = pd.read_parquet("data/volumes_hourly", columns=["site_id", "volume_mwh"],
                  filters=[("month", "=", "2026-03")])

# DuckDB : du SQL directement sur fichiers
duckdb.sql("SELECT * FROM 'data/volumes_hourly/*.*.parquet' LIMIT 5").df()
duckdb.sql("ATTACH 'data/risk.db' AS r (TYPE sqlite)")

# PySpark
from pyspark.sql import SparkSession, functions as F, Window as W
from pyspark.sql.types import *

spark = (SparkSession.builder.appName("risk")
        .config("spark.sql.session.timeZone", "UTC")
        .config("spark.driver.memory", "4g").getOrCreate())

v = spark.read.parquet("data/volumes_hourly")
b = (spark.read.option("header", True).option("sep", ";")
     .option("encoding", "ISO-8859-1").option("inferSchema", False)
     .csv("data/raw/bo_confirmations_20260724.csv"))
v.createOrReplaceTempView("vol")
spark.sql("SELECT COUNT(*) FROM vol").show()
```

1. SQL — noyau

Ordre logique d'évaluation

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

C'est cet ordre, et non l'ordre d'écriture, qui explique pourquoi un alias defini dans le SELECT est indisponible dans le WHERE, et pourquoi HAVING et WHERE ne sont pas interchangeable.

```
SELECT commodity, book,
       COUNT(*)           AS n,
       SUM(volume_mwh)     AS vol,
       AVG(price_eur_mwh)  AS px,
       MIN(trade_date)     AS first_trade,
       COUNT(DISTINCT counterparty) AS n_cpty
FROM trd_deal
WHERE status <> 'CANCELLED'
GROUP BY commodity, book
HAVING SUM(volume_mwh) > 1000
ORDER BY vol DESC
LIMIT 20;
```

Jointures

```
FROM a INNER JOIN b ON a.k = b.k -- intersection
FROM a LEFT JOIN b ON a.k = b.k -- tout a
FROM a FULL OUTER JOIN b ON a.k = b.k -- union (absent de SQLite ancien)
FROM a CROSS JOIN b -- produit cartésien

-- lignes de a sans correspondance dans b
SELECT a.* FROM a LEFT JOIN b ON a.k=b.k WHERE b.k IS NULL;
SELECT * FROM a WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.k=a.k);
SELECT k FROM a EXCEPT SELECT k FROM b;
```

CTE et sous-requetes

```
WITH base AS (
  SELECT * FROM trd_deal WHERE status='CONFIRMED'
), agg AS (
  SELECT book, SUM(volume_mwh) v FROM base GROUP BY book
)
SELECT * FROM agg WHERE v > 0;
```

Fenetrage

```
SELECT deal_id, version, price_eur_mwh,
       ROW_NUMBER() OVER (PARTITION BY deal_id ORDER BY version DESC) AS rn,
       RANK() OVER (PARTITION BY book ORDER BY volume_mwh DESC) AS rk,
       LAG(price_eur_mwh) OVER (PARTITION BY commodity ORDER BY trade_date) AS
       px_prev,
       SUM(volume_mwh) OVER (PARTITION BY book ORDER BY trade_date
       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum,
       AVG(price_eur_mwh) OVER (PARTITION BY commodity ORDER BY trade_date
       ROWS BETWEEN 4 PRECEDING AND CURRENT ROW) AS ma5
FROM trd_deal;
```

Une fonction de fenetre s'évalue *apres* le WHERE : on ne peut pas filtrer dessus dans la meme requete. Il faut une CTE, une sous-requete, ou QUALIFY (Spark, Databricks, pas SQLite).

Conditionnel, NULL, cast

```
CASE WHEN x > 0 THEN 'BUY' WHEN x < 0 THEN 'SELL' ELSE 'FLAT' END
COALESCE(a, b, 0) -- premier non NULL
NULLIF(a, 0) -- NULL si a=0 (evite la division par zero)
CAST(x AS REAL) -- SQLite : REAL, INTEGER, TEXT
```

x IS NULL / x IS NOT NULL -- jamais x = NULL

Chaines et dates

```
-- SQLite
substr(s,1,7) upper(s) lower(s) trim(s) replace(s,'a','b') length(s)
s LIKE 'D2%' s || '_suffixe'
date('now') strftime('%Y-%m', d) julianday(a)-julianday(b)
date(d,'+1 month') date(d,'start of month')

-- Spark SQL
substring(s,1,7) regexp_replace(s,['0-9'],'') lpad(s,10,'0')
date_format(d,'yyyy-MM') date_trunc('MONTH', d) add_months(d,1)
datediff(a,b) to_date(s,'dd/MM/yyyy') to_timestamp(s)
from_utc_timestamp(ts,'Europe/Paris') to_utc_timestamp(ts,'Europe/Paris')
```

Pièges SQL. COUNT(*) compte les lignes, COUNT(col) ignore les NULL. Une jointure sur une cle non unique multiplie les lignes sans avertissement : verifie la cardinalite avant, pas apres. En SQLite, 5/2 vaut 2 (division entiere). Un NOT IN sur une sous-requete contenant un NULL renvoie l'ensemble vide.

2. pandas

Inspection

```
df.shape df.dtypes df.info(memory_usage="deep") df.describe(include="all")
df.head() df.sample(5) df.columns.tolist()
df.nunique() df["c"].value_counts(dropna=False)
df.isna().sum() df.isna().mean().sort_values(ascending=False)
df.duplicated(subset=["k"]).sum()
df["k"].is_unique # test direct d'unicite de cle
df.memory_usage(deep=True).sum()/1e6
```

Selection et filtres

```
df.loc[df.commodity.eq("POWER"), ["deal_id", "volume_mwh"]]
df.iloc[0:5, 0:3]
df.query("volume_mwh > 100 and commodity == 'POWER'")
df[df.book.isin(["A","B"])] df[-df.book.isin([...])]
df[df.deal_id.str.startswith("D26")]
df[df.price_eur_mwh.between(50, 90)]
df.filter(like="date") df.filter(regex="^del")
```

Colonnes et chainage

```
df = df.assign(
    signed = lambda d: np.where(d.direction.eq("BUY"), 1, -1) * d.volume_mwh,
    dm = lambda d: pd.to_datetime(d.delivery_start).dt.to_period("M"),
)
df = df.rename(columns=str.lower).drop(columns=["tmp"])
df["c"] = df["c"].astype("float64") # "string", "category", "Int64"
df = df.astype({"volume_mwh": "float64", "deal_id": "string"})
```

Agregation

```
df.groupby(["commodity", "book"], dropna=False, observed=True).agg(
    n = ("deal_id", "size"), # size compte les NaN, count non
    nu = ("deal_id", "nunique"),
    vol = ("volume_mwh", "sum"),
    px_w = ("price_eur_mwh", lambda s: s.mean()),
    p95 = ("volume_mwh", lambda s: s.quantile(0.95)),
).reset_index()

# equivalent d'un HAVING
g = df.groupby("book").volume_mwh.sum()
g[g > 1000]

# valeur agreee ramenee au niveau ligne (pas d'agregation)
df["part"] = df.volume_mwh / df.groupby("book").volume_mwh.transform("sum")

# selection d'une ligne par groupe
df.sort_values("version").groupby("deal_id").tail(1)
df.loc[df.groupby("deal_id").version.idxmax()]
```

Jointures

```
a.merge(b, on="k", how="inner") # left / right / outer / cross
a.merge(b, left_on="deal_id", right_on="deal_ref", how="outer",
        indicator=True, suffixes=("_front", "_bo"),
        validate="one_to_one") # 1:l, 1:m, m:l, m:m -> leve si faux
pd.concat([a,b], axis=0, ignore_index=True)
a.set_index("k").join(b.set_index("k"), how="left")
pd.merge_asof(a.sort_values("t"), b.sort_values("t"), on="t",
              direction="backward") # jointure temporelle approchee
```

validate= et indicator=True sont les deux arguments les plus utiles de merge et les moins utilisés. Le premier transforme un bug silencieux en exception, le second materialise l'origine de chaque ligne.

Dates et fuseaux

```
s = pd.to_datetime(df.d, format="%d/%m/%Y", errors="coerce") # 'raise', 'coerce'
s.dt.year s.dt.month s.dt.hour s.dt.dayofweek s.dt.days_in_month
s.dt.to_period("M") s.dt.to_period("M").dt.to_timestamp()
s.dt.normalize() s.dt.floor("h") s.dt.strftime("%Y-%m")

s.dt.tz_localize("Europe/Paris", ambiguous="infer", nonexistent="shift_forward")
s.dt.tz_convert("UTC") s.dt.tz_localize(None)

pd.date_range("2026-01-01", "2026-12-31", freq="h", tz="Europe/Paris")
pd.bdate_range(a, b) pd.Timedelta(hours=1) pd.offsets.DateOffset(months=3)

df.set_index("ts").resample("D").sum() # 'h', 'D', 'MS', 'QS'
df.set_index("ts").asfreq("h") # revele les trous
```

Chaines

```
s.str.strip() .str.upper() .str.lower() .str.zfill(8) .str.pad(8, "left", "0")
```

```
s.str.replace(r"\s+", "", regex=True) s.str.extract(r"D(\d+)")
s.str.contains("D26", na=False) s.str.slice(0,7) s.str.split("_", expand=True)
s.str.len() s.str.match(r"D\d{7}$")
```

Manquants, doublons, remodelage

```
df.dropna(subset=["k"]) df.fillna({"v":0}) df.ffill() df.bfill()
df.drop_duplicates(subset=["k"], keep="last")
df[df.duplicated("k", keep=False)].sort_values("k") # voir tous les doublons

df.pivot_table(index="dm", columns="commodity", values="v",
                aggfunc="sum", fill_value=0, margins=True)
df.melt(id_vars=["k"], var_name="var", value_name="val")
df.stack() / df.unstack()
df.sort_values(["a","b"], ascending=[True,False])
```

Pieges pandas. `np.nan != np.nan` : compare avec `.isna()`, jamais avec `==`. Deux flottants issus de calculs différents ne sont presque jamais égaux : utilise `np.isclose(a,b,atol=...)`. Un `merge` sans valider sur une cle dupliquée gonfle silencieusement les lignes. `df[m]["c"] = x` ne modifie rien et lève un `SettingWithCopyWarning` : passe par `.loc[m,"c"] = x`. `groupby` ignore les NaN de la cle par défaut. Un `sum()` sur une colonne vide renvoie 0, pas NaN.

3. numpy

```
a = np.array([1.,2.,3.]) np.zeros(n) np.full(n,np.nan) np.arange(n)
a.shape a.dtype a.reshape(-1,1) np.concatenate([a,b])

np.where(cond, x, y)
np.select([c1,c2],[v1,v2], default=0)
np.clip(a, lo, hi) np.sign(a) np.abs(a) np.round(a,3)

a.sum() a.mean() a.std(ddof=1) a.min() a.argmax() a.cumsum()
np.nansum(a) np.nanmean(a) # ignorent les NaN
np.percentile(a,[5,50,95]) np.quantile(a,0.95)
np.diff(a) np.corrcoef(x,y) np.cov(x,y)
np.isnan(a) np.isfinite(a) np.isclose(a,b,rtol=1e-9,atol=1e-6)
np.unique(a, return_counts=True)
np.average(x, weights=w) # moyenne ponderee
```

Diffusion : une operation entre $(n,1)$ et $(1,m)$ produit (n,m) . Utile pour valoiriser une matrice position $\times$ prix, dangereux si les axes sont mal poses. Verifie systematiquement le `.shape` du resultat.

4. PySpark

Lecture, schema, exploration

```
df.printSchema() df.columns df.dtypes df.count()
df.show(10, truncate=False) df.limit(5).toPandas()
df.describe().show() df.summary("count","min","max").show()
df.explain(True) # plan d'execution
df.rdd.getNumPartitions()

schema = StructType([StructField("deal_ref", StringType(), True),
                      StructField("quantity", DoubleType(), True)])
spark.read.schema(schema).csv(path, header=True, sep=";")
```

Colonnes et filtres

```
df.select("a", F.col("b"), F.col("b").alias("c"))
df.selectExpr("a", "b * 2 AS b2")
df.filter((F.col("v") > 100) & (F.col("c") == "POWER")) # & | ~ et parentheses
df.where("v > 100 AND c = 'POWER'") # expression SQL
df.withColumn("signed", F.when(F.col("d")=="BUY", F.col("v"))
              .otherwise(-F.col("v")))
df.withColumnRenamed("a","b") df.drop("tmp")
df.withColumn("k", F.trim(F.upper(F.col("k"))))
```

Fonctions courantes

```
F.lit(0) F.coalesce(a,b) F.isNull(c) F.nanvl(a,b)
F.concat_ws("_", "a","b") F.substring("s",1,7) F.lpad("s",8,"0")
F.regexp_replace("s", r"s+", "") F.regexp_extract("s", r"D(\d+)", 1)
F.to_date("s","dd/MM/yyyy") F.to_timestamp("s") F.date_format("d","yyyy-MM")
F.date_trunc("MONTH","d") F.datediff(a,b) F.months_between(a,b)
F.from_utc_timestamp("ts","Europe/Paris") F.to_utc_timestamp("ts","Europe/Paris")
F.round(c,3) F.abs(c) F.greatest(a,b) F.least(a,b)
F.expr("CASE WHEN v>0 THEN 'B' ELSE 'S' END")
```

Agregation et jointures

```
(df.groupby("commodity","book")
 .agg(F.count("*").alias("n"),
      F.countDistinct("deal_id").alias("nu"),
      F.sum("volume_mwh").alias("vol"),
      F.avg("price").alias("px"),
      F.expr("percentile_approx(volume_mwh, 0.95)").alias("p95"))
 .filter(F.col("vol") > 1000) # equivalent du HAVING
 .orderBy(F.desc("vol")))
```

```
a.join(b, a.k == b.k, "inner") # inner, left, right, full/outer,
a.join(b, on="k", how="left_anti") # left_semi, left_anti, cross
a.join(F.broadcast(small), "k", "left") # force le broadcast si b est petit
a.unionByName(b, allowMissingColumns=True)
```

`left_anti` et `left_semi` n'ont pas d'équivalent direct en pandas et sont les jointures les plus utiles en controle de donnees : la premiere renvoie ce qui manque, la seconde filtre sans ramener de colonnes.

Fenetrage

```
w = W.partitionBy("deal_id").orderBy(F.desc("version"))
df.withColumn("rn", F.row_number().over(w))
df.withColumn("prev", F.lag("price",1).over(W.partitionBy("c").orderBy("d")))
wr = W.partitionBy("site_id").orderBy("ts").rowsBetween(W.unboundedPreceding, 0)
df.withColumn("cum", F.sum("v").over(wr))
```

Ecriture, cache, performance

```
(df.write.mode("overwrite").partitionBy("month").parquet("out/"))
df.cache() df.unpersist() # utile si le DataFrame est relu plusieurs fois
df.repartition(8,"site_id") df.coalesce(1)
spark.conf.set("spark.sql.shuffle.partitions", 16) # 200 par default, trop en local
```

Pieges PySpark. Tout est paresseux : `select`, `filter` et `join` ne calculent rien. Seules les actions (`count`, `show`, `collect`, `write`, `toPandas`) déclenchent l'exécution, et elles rejouent tout le plan à chaque fois. `collect()` et `toPandas()` rapatrient sur le driver : prévoir un `limit` ou une agregation avant. Utiliser `and/or/not` au lieu de `&/||/-` leve une erreur obscure. Une UDF Python casse l'optimiseur : chercher la fonction native avant. Le fuseau de session influence silencieusement toutes les conversions d'horodatage.

Table de correspondance

Operation	SQL	pandas	PySpark
Projection	SELECT a,b	df[["a","b"]]	df.select("a","b")
Filtre	WHERE v>100	df[df.v>100]	df.filter(F.col("v")>100)
Colonne calculee	SELECT v*2 AS w	df.assign(w=df.v*2)	df.withColumn("w",F.col("v")*2)
Renommage	AS	df.rename(columns={})	df.withColumnRenamed()
Agregation	GROUP BY k	df.groupby("k").agg()	df.groupBy("k").agg()
Filtre sur agregat	HAVING SUM(v)>0	g[g>0] apres agregation	.agg(...).filter(...)
Comptage distinct	COUNT(DISTINCT c)	df.c.nunique()	F.countDistinct("c")
Tri	ORDER BY v DESC	df.sort_values("v",ascending=False)	df.orderBy(F.desc("v"))
Jointure interne	INNER JOIN	merge(how="inner")	join(...,"inner")
Jointure externe	LEFT JOIN	merge(how="left")	join(...,"left")
Anti-jointure	WHERE NOT EXISTS	merge(indicator=True) puis filtre	join(...,"left_anti")
Union	UNION ALL	pd.concat([a,b])	a.unionByName(b)
Dedoublonnage	SELECT DISTINCT	df.drop_duplicates()	df.dropDuplicates()
Conditionnel	CASE WHEN	np.where/np.select	F.when().otherwise()
Valeur par defaut	COALESCE(a,b)	df.a.fillna(df.b)	F.coalesce("a","b")
Test de nullite	IS NULL	df.a.isna()	F.col("a").isNull()
Rang par groupe	ROW_NUMBER() OVER	groupby().cumcount()	F.row_number().over(W)
Decalage temporel	LAG(v) OVER	df.groupby("k").v.shift(1)	F.lag("v").over(W)
Agregat au niveau ligne	fenetre SUM() OVER	groupby().transform("sum")	F.sum().over(W)
Sous-chaine	substr(s,1,7)	s.str.slice(0,7)	F.substring("s",1,7)
Cast	CAST(x AS REAL)	df.x.astype(float)	F.col("x").cast("double")
Pivot	agregation conditionnelle	df.pivot_table()	groupBy().pivot().agg()
Limite	LIMIT 10	df.head(10)	df.limit(10)

Les trois colonnes ne sont pas trois vocabulaires a memoriser separement. Ce sont trois notations d'une meme algebre relationnelle : projection, selection, jointure, agregation, union. Si tu apprends la ligne plutot que la cellule, tu divises par trois le volume a retenir et tu deviens capable de traduire une operation que tu n'as jamais ecrite dans le troisieme langage. C'est exactement ce qu'on attend de toi sur Databricks, ou SQL et PySpark cohabitent sur les memes tables.

Patterns composes

A remplir par toi, au fil des missions. Un pattern par bloc :
le probleme, la regle, la requete retenue, et ce que tu as ecarte.

Probleme	Regle retenue / alternative ecartee
Normalisation de cle	
Classification des ecarts	
Selection de version courante	
Detection d' unite incoherente	
Controle de convention de signe	
Detection de fan-out de jointure	
Ventilation multi-periodes	
Journee a 23 / 25 heures	
Detection de trous par calendrier	
Alignement de fuseau	
Decomposition de P&L	
Seuil de materialite	

Cette page est le seul livrable qui compte vraiment. Les deux precedentes se retrouvent dans n'importe quelle documentation en trente secondes. Celle-ci, non : elle contient tes decisions, et une decision documentee avec son alternative ecartee est ce qui distingue un controleur des risques d'un executant.